

HazardLens AI — Case Study 1 Report

DS/CS553 — Machine Learning Development and Operations (MLOps)

Links for grading

Live product (Hugging Face Space) <https://huggingface.co/spaces/shyampatadia22/hazardlens-ai>

GitHub repository <https://github.com/shyampatadia/hazardlens-ai/>

Both repositories hold the same code. GitHub is the source of truth and the Space is kept in sync automatically by a GitHub Action on every push to `main`, which is described in the CI/CD section below.

a. Members

Shyam Patadia, Nischal Patel

b. Product

HazardLens AI takes a safety question written in plain English and answers it on the photo itself. You upload a picture of your workplace, you type something like *"Find anything blocking the emergency exit"*, and you get the same picture back with the objects that matter boxed and labelled.

The main idea here is that the user never has to name the objects they are looking for. A vision-language model looks at both the photo and the question and works out which objects are actually relevant. So when you ask about a blocked exit, it gives you the boxes and carts that are in the way, not the door itself, because the door is the thing being blocked and not the problem.

Who it is for: facility and warehouse safety officers, EHS staff, and site supervisors who do routine walkthrough inspections. We are very clear that this is a quick pre-check and not a certified inspection, and the interface says that directly.

What we added on top of a default Gradio app: a free-text inspection request instead of a fixed dropdown, a backend selector so you can pick Auto, Remote or Local, a confidence slider, and five outputs that show the annotated image, the labels it searched for, how many of each object it found, which model actually answered, and a status line.

c. Models

Role	Model	Where it runs
Remote VLM	<code>zai-org/GLM-5.3-Flash</code>	HF Inference API
Local VLM	<code>Qwen/Qwen3-VL-8B-Instruct</code>	Space ZeroGPU
Detector	<code>google/owlvit-base-patch32</code>	Space ZeroGPU

The app works in 2 stages. First the vision-language model turns the image and the goal into a list of object names. Then OWL-ViT takes that list and finds those objects in the image. Only the first stage switches between remote and local. OWL-ViT always runs on the Space no matter what you pick, and this matters a lot for the cost section later.

OWL-ViT is an open-vocabulary detector built on a ViT-B/32 backbone with CLIP-style image-text pretraining, with box and classification heads added on top. The useful part for us is that it matches against text embeddings instead of a fixed list of classes, so it can find objects it never saw during detection training. We need exactly that, because our label list is generated fresh for every request and we have no idea in advance what it will contain.

Qwen3-VL-8B-Instruct is an 8 billion parameter instruction-tuned multimodal model from Alibaba's Qwen team, and we run it through the Transformers `image-text-to-text` pipeline. **GLM-5.3-Flash** is a speed-focused multimodal model that we call over the HF Inference API, sending the image as a base64 JPEG.

We did not train or fine-tune anything. All 3 models are used as they are.

d. Performance

This is a real run against the deployed Space, 5 images, Auto mode:

Image	Latency	Result
<code>test_image_1</code>	64.65 s	failed
<code>test_image_2</code>	41.19 s	ok
<code>test_image_3</code>	40.26 s	ok
<code>test_image_4</code>	3.96 s	ok

Image	Latency	Result
test_image_5	12.22 s	ok

4 out of 5 succeeded. Mean 24.41 s, p50 26.24 s, p95 41.05 s.

The first request is the problem, not the app. Request 1 ran for 64.65 seconds and then timed out. That is the cold start, because the very first call has to get a ZeroGPU slot and then load an 8B model plus OWL-ViT before it can do any actual work. Every request after that one worked fine. So if we want an honest steady-state number we have to throw the first call away, and that is why our benchmark script now sends a separate warm-up request that does not count towards the statistics.

The latency splits into 2 groups. Some requests come back in about 4 seconds and others take about 40. That gap lines up with which backend answered. When the remote API handles it we get the 4 second result, and when it falls back to the local 8B model we get the 40 second result. So the remote model is roughly 10 times faster for the same work.

Both modes still use the GPU. This surprised us a bit. `analyze_image` is decorated with `@spaces.GPU(duration=120)`, and OWL-ViT runs on the Space regardless of which vision model you choose. So picking Remote does not make a request GPU-free, it only takes the 8B model off the GPU. Remote is cheaper on GPU, not free.

One thing we did deliberately is load both heavy models lazily, only when they are first needed. That makes the first request expensive and every later request cheap, and it also means our CI job can run the whole test suite without installing PyTorch at all.

e. Cost at 1,000 users

Let us assume 1,000 users and each of them does 10 inspections a month.

`1,000` users × `10` requests = `10,000` requests `per month`

Prices we are using (from the HF pricing page): HF Pro is \$9 a month and gives roughly 25 minutes of ZeroGPU a day, a dedicated T4 Small is about \$0.40 an hour, and the remote model costs roughly \$0.0002 per request.

Can ZeroGPU handle it?

This is the first thing we checked, and the answer is no, not even close.

25 min/day = 1,500 seconds a day
1,500 × 30 = 45,000 seconds a month

Now here is the catch. Our app declares `@spaces.GPU(duration=120)`, which means ZeroGPU reserves 120 seconds for every single request, even if that request actually finishes in 4 seconds. So the real question is how many 120 second blocks fit in our monthly allowance:

45,000 ÷ 120 = 375 requests a month

We need 10,000 and we can only do 375. That is under 4% of what we need. ZeroGPU is a prototyping tier and it simply cannot host this product at 1,000 users.

And this was not a theory, we actually hit it. An early benchmark run failed all 5 requests with `AppError: You have exceeded your ZeroGPU quota`, which is what made us go and look at the `duration=120` setting in the first place.

What a dedicated GPU costs

So we move to a dedicated Space GPU instead. Now the maths is just latency × number of requests.

Remote mode, about 4 seconds per request:

10,000 × 4 s = 40,000 seconds = 11 hours
11 × \$0.40 = \$4.40 GPU
10,000 × \$0.0002 = \$2.00 API

\$6.40 a month

Local mode, about 40 seconds per request:

10,000 × 40 s = 400,000 seconds = 111 hours
111 × \$0.40 = \$44.40 GPU
\$0.00 API

\$44.40 a month

That works out to about **\$0.0006 per inspection** on the remote path, which is less than a tenth of a cent.

What this tells us

The thing we did not expect is that **running the model ourselves is about 7 times more expensive**, not cheaper. It feels like self-hosting should be the free option because there is no API bill, but the API bill is only \$2 while the extra GPU time costs \$40. The model being 10 times slower is what does the damage, since we are paying for every one of those seconds.

Two honest caveats. First, a dedicated Space bills per hour that it is switched on, not per request, so if those 10,000 requests are spread thinly across the month we are still paying for a lot of idle time and the real cost per request would be worse than the numbers above. Second, the \$0.0002 per request figure is the weakest number here, because GLM-5.3-Flash actually bills per token and images use a lot of tokens, so that one should be checked properly before anyone relies on it.

f. Comments and concerns

Data privacy

This is the concern we take most seriously, and it is the one that would actually block a sale.

In remote mode we base64 encode the uploaded photo and send it to a third-party provider. Now think about what is in a workplace safety photo. There are usually employees in the frame, you can see the layout of the facility, and the whole point of the picture is that it is evidence of something being wrong. That is a bad combination to be shipping off to an external API.

A regulated customer, or one with a works council or a GDPR obligation, is going to ask us where that image goes and how long it is kept, and for the remote path our honest answer is that we do not fully control it. The local path does not have this problem at all, because the image never leaves the Space. So we think the local mode is not really a fallback, it is a separate product tier for customers who cannot use the remote one.

Depending on an external API

When we use the remote model, the availability, the latency, the pricing and even the model version are all decided by someone else. If the provider has an outage we are down. If they raise prices we pay it. And the one that worries us most is that if they silently update the model behind the same name, our label output could change and therefore our detections change, without us touching a single line of code. Our failover helps with the outage part but it does nothing about reproducibility.

Catching every exception is too broad

In Auto mode we wrap the remote call in a plain `try/except Exception`. That does catch real API failures, which is what we wanted, but it also catches our own bugs, malformed responses and

authentication mistakes, and then quietly relabels all of them as "remote unavailable" and sends the request to the slow local path.

The user never sees the real problem, they just see the app being slow. We actually experienced a version of this ourselves when our benchmark was recording errors but never printing them, and a quota failure looked like a generic failure for much longer than it should have. If we were taking this to production we would narrow this to the specific network and API exceptions.

Scalability

Two separate issues here. The quota ceiling from the cost section is the obvious one, and moving off ZeroGPU fixes it. The less obvious one is that the Space has no authentication and no rate limiting at all, so anyone who finds the URL can send requests and burn our GPU allowance. For a class project that is fine, but any real deployment needs per-user limits before it goes public.

Reliability

The cold start is a genuine user-facing risk and not just a benchmark artifact. The first request after the Space has been idle has to load 2 large models, and we measured that taking long enough to time out completely. The problem is that this lands on whoever shows up first, and a user whose very first attempt fails is probably not coming back to try again.

Safety and false negatives

We run OWL-ViT at a 0.05 threshold, which favours finding things over being precise. For a safety tool we think that is the right trade, because a false positive just means the inspector looks at a box that was fine, while a false negative means a blocked exit that nobody flagged. But it does mean the output needs a human to confirm it, and that is exactly why the "not a certified safety inspection" line in the interface matters and should stay there.

Secrets

This part we are happy with. The `.env` file is gitignored so the token never gets committed, and `HF_TOKEN` is supplied as a GitHub Actions secret and a Space secret instead of being in the code.

g. [LLM only] Strong LLM analysis

Model: Chatgpt Web version

Conversation: "<https://chatgpt.com/c/6aa87622-62a8-83e8-aaaf-b716e79dcbbe>"

Response: One meaningful **reliability and scalability improvement** would be to make the lazy model loading **thread-safe**.

Right now, both the local vision model and object detector are loaded only when their global variable is `None`.

If two users send requests at nearly the same time, both requests could see `_detector is None` and try to load the large model simultaneously. This could cause excessive GPU memory usage or even an **out-of-memory crash**.

Improvement: protect model initialization with a `threading.Lock`, so only one request can load each model at a time.

Why it matters: This makes the application more reliable under concurrent users and improves scalability without changing the actual AI functionality.

h. [LLM only] GLM Web Version

****Conversation**:** <https://chat.z.ai/s/f22b1010-2bf0-49da-80de-928a94332d43>

i. [Human only] Comparison of the two responses

Both of them suggested very good points, but I feel ChatGPT identified one key issue, which might cause a big production bug when deploying a product in real world where we are expecting multiple users to access the product in real time. And the GLM identified one bug which specifically focused on the instruction that we gave, and it followed instruction more closely, it was also a correct solution, but when I compare the 2 responses ChatGPT identified one key bug and also it gave us very concise answer.

One key improvement is that none of the LLMs suggested any improvements on the product itself, they gave the issue with the code but not with the idea of the product itself, which was very surprising to notice, given that we asked it to look for improvements in every direction.

j. [Human only] Deployment decision

So there are 2 things to consider, if we are deploying a product which collects the data which is very sensitive (medical records, financial details, legal documents), then I would definitely prefer to host the model in our own infrastructure no matter what is the cost, because at the end of the day, the user data is the most aspect, which is of top priority.

But the stronger models are very useful for normal day to day tasks, like for example perplexity is a very good example of a product, that I would definitely prefer to use compared to regular google search, or let's say for literature review suppose I am working on a research project, I would definitely like to use the power of agentic harness behind these large LLMs which can help me speed up my work or make search faster for me.

Extra credit: Adaptive failover

This is implemented in `expand_goal()` in `helpers.py`.

Detecting the failure. In Auto mode the remote call sits inside a try/except, so a timeout (we set a 30 second limit), a rate limit, an outage or an auth failure all trigger the fallback.

Routing automatically. When that happens the request is sent to the local Qwen3-VL model straight away. The user does not have to do anything or even know it happened.

Showing which model answered. The "Vision model used" box in the interface shows either Remote: ..., Local: ... OR Local fallback: We deliberately made the fallback string different so you can tell an automatic switch apart from a manual choice, and the actual error message gets passed through to the status field so the user can see why it switched.

Demonstrating it. `test_model_modes_and_auto_fallback` in `tests/test_hazardlens.py` forces the remote path to raise an exception and then checks that the labels came from the local model, that the backend string has the fallback prefix, and that the error text reached the user. It also checks that if you explicitly pick Remote or Local it does *not* fail over, because if someone deliberately chose a backend we should not quietly move them off it.

Advantages. Availability goes up because the 2 backends fail independently, so a provider outage no longer takes the product down. The user gets a slower answer instead of an error page. And there is a nice side effect, which is that the fallback path is also the private one, so an outage actually moves us to a more private setup rather than a less private one.

Disadvantages. The fallback is around 10 times slower, so we are buying availability with latency that the user did not ask for. It also uses much more ZeroGPU quota per request, which means a long remote outage could drain the daily allowance and then take down both paths, so the failover can end up causing the next failure. And because the 2 models are different, they can return different labels for the same image, so results are not reproducible across a failover.

CI/CD and notifications

`.github/workflows/deploy.yml` runs on every push to `main` and has 3 jobs that depend on each other:

1. **test** — sets up Python 3.11, installs the test dependencies and runs pytest. It installs no PyTorch at all, which only works because we defer the `torch` and `transformers` imports into the loader functions.
2. **deploy** — has `needs: test`, so a failing test blocks the deployment. It syncs the repo to the Space using `huggingface/hub-sync` with the `HF_TOKEN` secret.

3. **notify** — has `if: always()` so it reports either way, and posts the result to Discord through the `DISCORD_WEBHOOK` secret.

The tests cover label parsing, backend selection and failover, the duplicate detection filter, the end-to-end result flow and the benchmark statistics. Every model call is mocked, so CI never touches an API or asks for a GPU.

So the flow is that we push to GitHub, the action runs the tests, and if they pass it pushes the same code to the Space and tells us on Discord either way. This is what keeps the evil CEO happy, because GitHub stays the source of truth and nobody has to remember to update the Space by hand.

- Workflow file: <https://github.com/shyampatadia/hazardlens-ai/blob/main/.github/workflows/deploy.yml>
- Repository: <https://github.com/shyampatadia/hazardlens-ai/>
- Deployed Space: <https://huggingface.co/spaces/shyampatadia22/hazardlens-ai>

How the pitch video was made

We generated the 2:30 pitch video from code instead of editing it in a video editor, so it rebuilds from the repository. The source is in `pitch/`.

Layer	Tool
Video framework	Remotion 4.0.409 (React + TypeScript)
Runtime	Node 24, rendered through headless Chromium
Demo data	<code>google/owlvit-base-patch32</code> run locally on CPU
Voiceover	<code>edge-tts</code> , Microsoft neural TTS, <code>en-US-AndrewNeural</code>
Audio and mux	FFmpeg 8.0

How it works. Each of the 8 scenes is a React component in `pitch/src/scenes/`, animated with Remotion's `spring()` and `interpolate()` based on the current frame number. `Pitch.tsx` puts them in order with an 18 frame cross-fade between each one, which gives 4,500 frames at 1920×1080 and 30 fps. Then `npx remotion render` drives a headless Chromium, screenshots every frame and encodes it to H.264.

The demo boxes are real, not drawn by hand. Before rendering we ran OWL-ViT locally on CPU against `test_images/test_image_2.jpg`, using the labels a vision-language model returns for the exit-blocking prompt (`box`, `cart`, `pallet`) at a 0.22 threshold, and passed the output through the

same `remove_duplicate_detections` function the app uses. The 4 boxes in the video at 0.32/0.32/0.25/0.24 confidence are that actual result, saved to `pitch/public/detections.json` and drawn at the right scale by the scene. Running the detector locally also meant that making the video cost us no ZeroGPU quota at all. The performance and cost numbers on screen are the same measured ones from sections (d) and (e).

Voiceover. `pitch/narrate.py` holds one narration line per scene along with that scene's start time and how long it has. It generates each line, measures it with `ffprobe` and checks it actually fits, and if a line is too long it regenerates it slightly faster instead of letting it run into the next scene. The clips are then placed on a silent track at their scene offsets with `adelay` and `amix`, held out to full length with `apad`, normalised to the -16 LUFS web speech standard with `loudnorm`, and muxed onto the video with the video stream copied rather than re-encoded.

Editing a line and re-running takes about 20 seconds and needs no re-render. `pitch/NARRATION.md` has the same script written out for recording a human voice, which is what we would use for a real investor meeting.

Citations

Models: [GLM-5.3-Flash](#) · [Qwen3-VL-8B-Instruct](#) · [OWL-ViT](#) · Minderer et al., *Simple Open-Vocabulary Object Detection with Vision Transformers*

Docs: [Spaces](#) · [Spaces with GitHub Actions](#) · [HF Pricing](#) · [Discord webhooks](#)

Video tooling: [Remotion](#) · [edge-tts](#) · [FFmpeg](#)

LLM assistance: Claude Opus 5 (Anthropic) via Claude Code — diagnosing the ZeroGPU authentication failure, rewriting `testing_performace.py`, building the cost model, writing the Remotion pitch video and its narration script. Performance figures in (d) come from actual runs against the deployed Space, and the detection boxes in the video come from a local OWL-ViT run. The voiceover in the video is synthetic speech, not a recorded human voice.